Build Guide

Guía paso a paso: construir picoSargon v0.2.0 en Visual Studio Code (Windows 11)

El objetivo es partir de una carpeta de proyecto, configurar Visual Studio Code para utilizar las herramientas que ya tenés instaladas y terminar con uno de estos archivos:

- Raspberry Pi Pico W (RP2040): `build-picow/picoSargon.uf2`
- Raspberry Pi Pico 2 W (RP2350): `build-pico2w/picoSargon.uf2`

Voy a utilizar el proyecto real que generamos, sin crear otro programa de ejemplo ni reemplazar su `CMakeLists.txt`.

Proyecto fuente picoSargon v0.2.0

C++17, CMake, Pico SDK y herramientas Python

Descargar ZIP del proyecto

Paso 1. Crear la carpeta de trabajo

Te propongo utilizar la misma organización de proyectos que venimos empleando:

```
C:\Users\pedro\github\Bobby\picoSargon\
```

1. Creá esa carpeta desde el Explorador de Windows.
2. Extraé allí el contenido del ZIP.
3. El ZIP contiene una carpeta interior denominada `picoSargon_v0.2.0`. Podés renombrarla a `picoSargon` o mover su contenido a la carpeta definitiva.

La estructura final debe quedar así:

```
C:\Users\pedro\github\Bobby\picoSargon\
│
├── .vscode\
├── assets\
│   └── grumpy.bin
├── include\
├── reference\
├── src\
├── tests\
├── tools\
├── CMakeLists.txt
```

```
├── pico_sdk_import.cmake
└── README.md
```

Es fundamental que `CMakeLists.txt` esté directamente en la raíz del proyecto, no un nivel por debajo.

Paso 2. Abrir el proyecto existente en VS Code

1. Abrí Visual Studio Code.
2. Elegí File → Open Folder (Archivo → Abrir carpeta).
3. Seleccioná `C:\Users\pedro\github\Bobby\picoSargon`.
4. Confirmá que la carpeta sea de confianza si VS Code lo pregunta.
5. Abrí la terminal integrada con Terminal → New Terminal.

No utilices todavía «New Pico Project» de la extensión Raspberry Pi Pico. Ese asistente crea otro proyecto y podría reemplazar nuestra configuración CMake. Ya tenemos un proyecto completo: debemos abrirlo e importar su configuración existente.

Paso 3. Comprobar las extensiones

Aunque ya estén instaladas, verificá que estén habilitadas en el perfil actual de VS Code.

Extensión	Finalidad
Raspberry Pi Pico	Integración del Pico SDK y herramientas ARM
CMake Tools	Configurar y construir el proyecto
C/C++ de Microsoft	IntelliSense y depuración C++
Python de Microsoft	Ejecutar los generadores de configuración

image

La extensión Raspberry Pi Pico y CMake Tools deben trabajar con el mismo SDK y compilador, para evitar configuraciones incompatibles.

Paso 4. Identificar las rutas reales del SDK y compiladores

En una instalación Windows realizada mediante la extensión Raspberry Pi Pico, suele existir una estructura parecida a esta:

```
C:\Users\pedro\.pico-sdk\
    sdk\
        2.x.x\
    toolchain\
        14_2_Rel1\
        bin\
    cmake\
    ninja\
```

Esta estructura es ilustrativa. Las carpetas exactas dependen de la instalación; no copies los nombres de versión sin comprobarlos.

En VS Code, abrí la configuración de la extensión Raspberry Pi Pico y localizá las rutas de:

- Pico SDK
- ARM GNU Toolchain
- CMake
- Ninja

También podés ejecutar estas comprobaciones en la terminal PowerShell:

```
Get-Command cmake
Get-Command ninja
Get-Command arm-none-eabi-gcc
Get-Command python
```

Y verificar versiones:

```
cmake --version
ninja --version
arm-none-eabi-gcc --version
python --version
```

Si alguna herramienta no se reconoce, no significa necesariamente que no esté instalada: puede faltar su directorio en la variable `PATH`.

Importante: qué debe contener `PICO_SDK_PATH`

Tiene que apuntar a la raíz del repositorio SDK, donde existen:

```
pico-sdk\
    external\
        pico_sdk_import.cmake
    src\
    lib\
    CMakeLists.txt
```

No debe apuntar a `external`, a `src` ni a la carpeta que contiene todas las versiones del SDK.

Paso 5. Vincular el SDK en la terminal de VS Code

Para evitar depender de configuraciones globales de Windows, inicialmente recomiendo usar variables de entorno locales a la terminal PowerShell.

En el siguiente ejemplo, reemplazá las rutas de ejemplo por las que hayas comprobado en el paso anterior:

```
$env:PICO_SDK_PATH = "C:\Users\pedro\.pico-sdk\sdk\2.2.0"
$env:Path = "C:\ruta\real\arm-toolchain\bin;$env:Path"
```

Si CMake y Ninja tampoco están en el `PATH`, agregá sus directorios correspondientes del mismo modo:

```
$env:Path = "C:\ruta\real\cmake\bin;$env:Path"
$env:Path = "C:\ruta\real\ninja;$env:Path"
```

Estas variables permanecerán activas solamente durante esa sesión de PowerShell.

Verificá que el SDK esté correctamente referenciado:

```
Test-Path "$env:PICO_SDK_PATH\external\pico_sdk_import.cmake"
Test-Path "$env:PICO_SDK_PATH\src"
```

Ambos comandos deben devolver `True`.

Si el SDK está instalado mediante la extensión y no sabés dónde está, podés inspeccionar las carpetas de configuración con:

```
Get-ChildItem "$HOME\.pico-sdk" -Directory
```

Si la extensión utiliza otra ubicación, empleá esa ruta.

Un detalle: si CMake Tools había configurado previamente el proyecto con otro compilador, habrá que borrar el directorio de compilación correspondiente. No conviene reutilizar una caché generada para otra arquitectura.

Paso 6. Generar la configuración privada del firmware

En la raíz de `picoSargon`, usando la terminal de VS Code, ejecutá:

```
python tools/generate_config.py --ssid bolSPA_EXT --port 6110 --timeout-ms 0 --
plies 2
```

El programa solicitará la contraseña Wi-Fi sin mostrarla en pantalla.

La configuración generada quedará en:

```
include\generated_config.hpp
```

Contendrá estos parámetros:

Parámetro	Valor del ejemplo
`SSID`	`bolSPA_EXT`
`PASSWORD`	Contraseña introducida
`PORT`	`6110`
`TIMEOUT_MS`	`0` — sin límite temporal
`PLIES`	`2`

Verificá su existencia:

```
Test-Path .\include\generated_config.hpp
```

Debe devolver `True`.

El archivo generado contiene la contraseña en texto claro como constante C++, por lo que no debe incorporarse a Git ni compartirse. El proyecto ya lo excluye mediante `.gitignore`.

Paso 7. Generar los pesos Grumpy embebidos

El firmware no leerá `grumpy.bin` de un sistema de archivos MicroPython. Los pesos quedarán integrados en el binario C++.

Ejecutá:

```
python tools/embed_grumpy.py
```

Esto genera o actualiza:

```
include\generated_grumpy.hpp
```

Comprobá:

```
Test-Path .\include\generated_grumpy.hpp
```

Debe devolver `True`.

A esta altura tenemos el proyecto C++ y los dos encabezados generados que necesita la compilación.

Paso 8. Configurar la compilación para Pico W

Vamos a compilar primero para la Pico W original (RP2040), que ya tenés disponible. Después repetiremos el proceso para Pico 2 W.

Desde la terminal integrada de VS Code, ubicado en la raíz del proyecto:

```
cmake -S . -B build-picow -G Ninja `
    -DPICOSARGON_PICO=ON `
    -DPICO_BOARD=pico_w `
    "-DPICO_SDK_PATH=$env:PICO_SDK_PATH" `
    -DCMAKE_BUILD_TYPE=Release
```

Qué significa cada opción:

Opción	Función
`-S .`	Carpeta del código fuente
`-B build-picow`	Carpeta donde se generará la compilación
`-G Ninja`	Generador de compilación
`PICOSARGON_PICO=ON`	Construir firmware y no ejecutable para PC
`PICO_BOARD=pico_w`	Seleccionar hardware RP2040 con Wi-Fi
`PICO_SDK_PATH`	Ruta exacta al SDK
`Release`	Compilación optimizada

Si todo funciona, CMake identificará el Pico SDK, los compiladores ARM y los componentes necesarios. Deberías ver al final un mensaje similar a:

```
-- Configuring done
-- Generating done
-- Build files have been written to: .../build-picow
```

No continúes con el siguiente paso si CMake informa errores.

Paso 9. Compilar el firmware

Ejecutá:

```
cmake --build build-picow --parallel 4
```

El proceso compilará las fuentes de `src/`, enlazará el motor con Pico SDK y generará los archivos de salida.

El artefacto esperado es:

```
build-picow\picoSargon.uf2
```

Verificá:

```
Test-Path .\build-picow\picoSargon.uf2
```

Si devuelve `True`, comprobá su tamaño:

```
Get-Item .\build-picow\picoSargon.uf2 |
    Select-Object Name, Length, LastWriteTime
```

En este punto habremos construido el firmware para Pico W, aunque todavía faltará validarlo sobre hardware real.

Paso 10. Compilar también para Pico 2 W

La Pico 2 W usa el RP2350, de modo que necesitamos una compilación independiente.

Ejecutá:

```
cmake -S . -B build-pico2w -G Ninja `
    -DPICOSARGON_PICO=ON `
    -DPICO_BOARD=pico2_w `
    "-DPICO_SDK_PATH=$env:PICO_SDK_PATH" `
    -DCMAKE_BUILD_TYPE=Release
```

Luego:

```
cmake --build build-pico2w --parallel 4
```

El resultado esperado es:

```
build-pico2w\picoSargon.uf2
```

La compilación para Pico 2 W requiere una versión del Pico SDK que reconozca `pico2_w` y una cadena de herramientas compatible con RP2350. No copies el `.uf2` de Pico W a Pico 2 W ni viceversa.

Paso 11. Hacer las compilaciones desde la interfaz de VS Code

Una vez que comprobamos que CMake funciona desde la terminal, podemos dejar de escribir los comandos manualmente.

El proyecto ya incluye `.vscode/tasks.json`, con las tareas de construcción.

1. Abrí Terminal → Run Task....
2. Elegí `picoSargon: Configure Pico W`.
3. Una vez finalizada, elegí `picoSargon: Build Pico W`.
4. Para Pico 2 W, utilizá `picoSargon: Configure Pico 2 W` y después `picoSargon: Build Pico 2 W`.

Tareas disponibles en VS Code

picoSargon: Generate private config

picoSargon: Embed Grumpy

picoSargon: Configure Pico W

picoSargon: Build Pico W

picoSargon: Configure Pico 2 W

picoSargon: Build Pico 2 W

Atención: las tareas existentes invocan `cmake` y `ninja` por nombre y no incluyen rutas absolutas. Funcionarán si VS Code hereda un entorno donde esas herramientas, el compilador ARM y `PICO_SDK_PATH` están correctamente definidos.

Si no los encuentra, primero debemos arreglar las rutas; no conviene duplicar el SDK ni instalar otro compilador.

La extensión Raspberry Pi Pico también dispone de su propio mecanismo de configuración y compilación de proyectos. Podemos utilizarlo posteriormente, pero inicialmente prefiero las tareas CMake existentes porque sus argumentos están explícitos y son reproducibles.

Paso 12. Resolver los errores de configuración más comunes

Error	Causa probable	Corrección
`PICO_SDK_PATH not defined`	No está configurada la ruta	Revisar paso 5
`arm-none-eabi-gcc not found`	Compilador fuera del PATH	Añadir directorio `bin`
`ninja not found`	Ninja fuera del PATH	Localizar ejecutable
`Unknown board pico2_w`	SDK antiguo o placa no soportada	Seleccionar SDK actualizado
`generated_config.hpp: No such file`	No se generó la configuración	Ejecutar paso 6
Error de librería `pico_cyw43_arch_lwip_poll`	SDK o submódulos incompletos	Revisar SDK y submódulos

Error	Causa probable	Corrección
`CMakeCache.txt` con rutas anteriores	Caché de otra configuración	Eliminar solo la carpeta `build-*` afectada

Una advertencia específica sobre nuestra v0.2.0

El código C++ de `pico_main.cpp` utiliza excepciones estándar (`try`/`catch`), `std::string` y otras características del runtime C++.

La primera compilación cruzada todavía no fue validada. Si el SDK o el toolchain tienen deshabilitadas las excepciones, podríamos encontrar un error de compilación. En ese caso corresponde corregir específicamente la configuración o el manejo de errores de ese archivo; no debemos modificar arbitrariamente el algoritmo de ajedrez.

Además, compilar el firmware no garantiza todavía que la comunicación Wi-Fi/TCP funcione sin fallos: esa capa requiere una prueba física.

Paso 13. Cargar el `.uf2` en la Pico

Cuando el firmware para tu placa haya compilado sin errores:

1. Desconectá la Pico W del USB.
2. Mantené presionado el botón BOOTSEL.
3. Conectá nuevamente el cable USB manteniendo el botón presionado.
4. Soltá BOOTSEL cuando Windows detecte la unidad `RPI-RP2`.
5. Copiá el archivo `build-picow\picoSargon.uf2` a esa unidad.
6. La placa se reiniciará ejecutando el nuevo firmware nativo.

En PowerShell, si la unidad montada fuera `E:`, por ejemplo:

```
Copy-Item .\build-picow\picoSargon.uf2 E:\
```

Usá la letra real asignada por Windows.

Precaución: al instalar firmware C++ dejará de arrancar MicroPython. Conservá los respaldos de los archivos anteriores para poder recuperar ese entorno si fuera necesario.

Paso 14. Comprobar el arranque y la telemetría

Abrí el monitor serial de VS Code y seleccioná el nuevo puerto COM que exponga la placa.

Esperamos una secuencia semejante a:

```
picoSargon C++ v0.2.0 BOOT
WiFi connecting SSID=bolSPA_EXT
UCI ready TCP port 6110 IP 192.168.68.xxx
```

La IP vendrá del servidor DHCP y puede no coincidir con la utilizada anteriormente.

Comprobá también el LED: fijo durante inicialización, alternancia cada segundo cuando está listo, cada tres segundos durante búsqueda y rápida ante error fatal.

Cuando aparezca el mensaje de TCP listo, desde Windows ejecutá:

```
python tools/tcp_uci_test.py 192.168.68.xxx --port 6110 --timeout 180
```

Reemplazá la IP por la que muestre el monitor serie.

El cliente de prueba permite un timeout prolongado porque estamos trabajando con búsquedas de 2 ply sin límite temporal.

Secuencia resumida de comandos

Cuando tengamos las rutas correctamente configuradas, toda la compilación de Pico W queda reducida a:

```
# Desde la raíz de picoSargon

python tools/generate_config.py --ssid bolSPA_EXT --port 6110 --timeout-ms 0 --
plies 2

python tools/embed_grumpy.py

cmake -S . -B build-picow -G Ninja `
    -DPICOSARGON_PICO=ON `
    -DPICO_BOARD=pico_w `
    "-DPICO_SDK_PATH=$env:PICO_SDK_PATH" `
    -DCMAKE_BUILD_TYPE=Release

cmake --build build-picow --parallel 4
```

Resultado:

`build-picow\picoSargon.uf2`

Cómo propongo que avancemos

Primero completaría los pasos 1 al 5, hasta verificar que VS Code reconoce el SDK y el compilador ARM que ya están instalados.

No necesitamos instalar nuevamente nada.

El primer dato que me interesa es el resultado de estos comandos en la terminal de VS Code:

```
Get-Command cmake,ninja,arm-none-eabi-gcc
```

```
echo $env:PICO_SDK_PATH
```

Con esas rutas podremos configurar correctamente el proyecto y resolver cualquier problema específico de Windows antes de intentar generar el primer `.uf2`.